The Open Source Audit — Git

Course: Open Source Software | **Unit Coverage:** 1–5 | **Max Marks:** 100

Student Name Siddharth Jain
Registration Number 24MIM10220 Slot F11 Date of Submission 31/03/2026 Chosen Software Git

---

Introduction

I picked Git for this project because it's something I actually use. Not once in a while, but literally every day. Every time I push code or make a branch or deal with some merge conflict at 2 AM, Git is running underneath all of it. I never really thought about where it came from though. Like, I knew Linus Torvalds was involved somehow, but I didn't know the details. Working on this project forced me to actually dig into the history, and the story turned out to be way more interesting than I was expecting.

The report has four main parts. Part A goes into Git's origin story, its license, and some ethical questions around open source. Part B is the technical side, where exactly Git's files live on a Linux system and how permissions work. Part C covers the ecosystem. Part D is a comparison against a proprietary alternative. There are also five shell scripts that I wrote and tested, which are documented at the end.

Part A — Origin and Philosophy (Units 1 & 2)

A1. How Git Came to Exist

The story of Git starts with a fight. Not a technical one — a legal one.

Back in the early 2000s, the Linux kernel was using a tool called BitKeeper for version control. It's the biggest open-source project out there, so version control is a big deal for them. BitKeeper was proprietary. It was owned by a company called BitMover, and they'd given the Linux developers a free license to use it. The catch? Nobody was allowed to reverse-engineer how it worked.

That worked fine for about three years. Then in 2005, a developer named Andrew Tridgell (who'd also built Samba) tried to reverse-engineer the BitKeeper protocol. BitMover didn't like that, so they pulled the free license. Just like that, the entire Linux kernel project lost its version control system.

Think about what that means for a second. You've got hundreds of developers across the world working on the same codebase, and overnight the tool they use to coordinate all of that just disappears. The alternatives that existed at the time, like CVS and

Subversion, were centralized. Meaning everyone had to connect to one central server to commit changes. Server goes down? Nobody can work. For the Linux kernel, with developers spread across every timezone, that kind of setup was way too slow and way too fragile.

So Linus Torvalds, the guy who created Linux, basically said "never again" to proprietary tools and sat down to write his own version control system. From scratch. And it took him about two weeks. I still can't wrap my head around that. The tool that like 95% of developers use today was built in two weeks. Most of us can't even finish a homework assignment that fast.

He had a few things he specifically wanted:

1. It had to be fast. Branching and merging had to happen instantly, not take minutes over a network.
2. It had to be distributed. Every developer gets a full copy of the project history on their own machine. No single server that can bring everything down.
3. It had to protect data integrity. Git uses SHA-1 hashes for everything — every file, every commit. Change one byte and the hash is different. You can't tamper with history without it being obvious.
4. It had to handle lots of parallel work. The kernel has thousands of people working on different things at the same time. Git needed to handle thousands of branches without breaking a sweat.

He named it "Git" — which is British slang for a stupid or annoying person. When asked about it, Torvalds said something like "I name all my projects after myself. First Linux, now Git." I thought that was pretty funny.

It spread pretty quickly after that. GitHub launched in 2008 and suddenly Git wasn't just for kernel hackers anymore, normal developers could use it too. The Stack Overflow Developer Survey puts Git usage at over 95% today. For something that started because of a licensing fight, that's kind of crazy.

The thing that stuck with me from this whole story is the pattern. The Linux community put its trust in a proprietary tool. One business decision later, they were locked out. That's it. And instead of negotiating or shopping around for another proprietary replacement, Torvalds just wrote something better and gave it away. If there's a takeaway here, it's probably don't build your workflow on top of something you don't control.

A2. The License — What It Actually Says

Git uses the GNU General Public License, version 2 — GPLv2. It's the same license the Linux kernel uses, and it's been around since 1991.

I went and read the actual license text on gnu.org. It's not that long, but the language is dense. Here's what it boils down to. **The Four Freedoms**

The Free Software Foundation (Richard Stallman's organization) says free software has to give you four specific freedoms:

- **Freedom 0** — You can run the software for whatever you want. Nobody gets to say "you can use this, but not for that."
- **Freedom 1** — You can look at the source code and change how it works. Obviously this requires the source code to be available.
- **Freedom 2** — You can give copies to other people.
- **Freedom 3** — You can distribute your modified versions too, so other people benefit from your improvements.

Git's license covers all four of these. Download it, read the code, change whatever you want, hand your version to anyone. No restrictions on any of that. But. There is one condition.

The copyleft part

GPLv2 is what's called a "copyleft" license. If you take Git, modify it, and then distribute your modified version, your version also has to be GPLv2. You can't take Git, make it better, and then close the source and sell it as some proprietary product. That's the whole point of copyleft — it makes sure the code stays open, no matter who touches it.

Can a company modify Git and sell it without sharing code?

Short answer: no. If they distribute a modified version, they have to include the source code and license it under GPLv2. They CAN charge money for it — the GPL doesn't say software has to be free of cost — but they can't hide the code. Anyone who gets a copy also gets the right to see, modify, and share the source.

GPL vs MIT

The MIT license is way more relaxed. Basically it says: do whatever you want, just keep the copyright notice. A company could take MIT-licensed code, modify it, close the source completely, and sell it. Nobody could stop them.

GPL says no to all of that. It's stricter, but the strictness is protecting the users, not the developer. With MIT, you're kind of hoping companies will do the right thing. With GPL, hope isn't part of the equation. The license makes them do it.

If I had to choose for my own project, I think it'd depend on what I'm building. For some small library where I want maximum adoption, MIT makes sense because companies aren't scared of it. But for a major project that I want to stay free forever? GPLv2. I don't want to see my work locked up inside some corporate product. I think Torvalds made the right call with Git.

"Free as in beer" vs "free as in freedom"

People use "free" to mean two very different things. "Free as in beer" = you don't pay for it. "Free as in freedom" = you have the legal right to study it, change it, and share it.

Git is both. You pay nothing to download it, and you also have full access to every line of code. You can change whatever you want. But if you had to pick one, the freedom matters more than the price. Like, imagine Git was free to download but the source was locked. You could use it, yeah, but you couldn't fix bugs, you couldn't learn how it works inside, you couldn't adapt it for something it wasn't designed for. That would be free beer with no freedom. GPL prevents that situation.

A3. Ethics of Open Source

Should everything be open source?

I went back and forth on this one for a while. My first instinct was obviously yes, everything should be open. More eyes on the code means fewer security holes. Developers stop wasting time rebuilding stuff that already exists. Students who can't afford expensive software licenses get access to real tools. All of that sounds good on paper.

But then you think about edge cases. Medical device software, for example. That goes through heavy regulatory approval. You really want random people modifying the code running on a heart monitor? That could get someone killed. Or take video games. Studios pour years of work and crores of rupees into game engines and art. Force them to open source all of it and the business model just collapses. A lot of games we enjoy would never get made.

Also, defense. I don't think anyone's seriously arguing that military systems should have their source code published where anyone can read it.

So I guess my position is: open source should be the default for infrastructure stuff. Operating systems, databases, compilers, web servers, version control. These things are too important for one company to hold hostage. But making it mandatory for literally everything? No. There are cases where closed source makes sense. The important thing is having a real reason for it, not just "we want more money."

Companies making money from free community work

This one genuinely bothers me. Red Hat makes money selling enterprise Linux support. Amazon takes open-source databases and sells them as cloud services. Meta uses Linux on every single server they run. These are companies making billions, and a huge chunk of the software they rely on was written by volunteers who didn't see a paisa.

Is that fair? I think it depends. Red Hat actually employs a lot of kernel developers and pushes code back upstream. Google created Kubernetes and TensorFlow and released them as open source. Those companies are giving back, so OK, fine. But there are other companies that just take. They grab whatever open-source tools they need, build a product, charge customers, and contribute zero back to the projects they depend on. Not

code, not money, not even a bug report. That's just freeloading. The licenses technically allow it, sure. But it still doesn't sit right.

I've read about newer licenses like the Business Source License and Server Side Public License that are trying to stop companies from just reselling open-source projects as cloud services. I don't know if they'll catch on, but the frustration in the community is real.

Standing on shoulders

In software this phrase is literally true. Any app I write uses libraries someone else made. Those libraries got compiled by compilers that other people wrote. The compilers run on an operating system that thousands of people contributed to. It's layers all the way down.

Git is a good example of this. It's C code, compiled with GCC, running on Linux. All open source. GitHub runs on open-source web servers. Even the developers working on Git are using open-source text editors and debuggers. Everything in the chain is open.

So does open source help innovation or does it make people lazy? I think it helps, pretty clearly. Imagine if Torvalds had to write his own compiler and his own OS before he could even start working on Git. We'd probably still be using Subversion. Open source means you don't have to solve problems that are already solved. You just build on top of what's there and focus on the new stuff.

Part B — Linux Footprint (Unit 2)

How You Install It

On Ubuntu (which is what I used), installing Git is one command:

```
sudo apt update sudo
apt install git
```

On Fedora or Red Hat, it's:

```
sudo dnf install git
```

You can also build it from source if you clone the repo at `https://github.com/git/git` and run `make`. I tried this just to see — it works fine but takes way longer and you have to deal with dependencies yourself. The package manager handles all that for you, so there's really no reason to compile from source unless you want to test a specific version.

Where the Files Go

After installation on Ubuntu, Git's files end up spread across the standard Linux directory structure:

Path	What's there
`/usr/bin/git`	The main Git binary — this is what actually runs when you type `git`
`/usr/lib/git-core/`	Internal helper tools like `git-merge`, `git-rebase`, etc.
`/usr/share/doc/git/`	Documentation and examples
`/usr/share/man/`	Man pages (so `man git` works)
`~/.gitconfig`	Your personal config — name, email, preferred editor, etc.
`~/.config/git/`	Another spot for user configs (less commonly used)
`.git/` (in any repo)	The actual repository data — all your commits, branches, objects

Users and Groups

Git doesn't run as a background daemon, so there's no dedicated "git" system user. It just runs as whoever called it. If I type `git commit` as `grahu`, the process runs with `grahu`'s permissions. If I use `sudo git commit`, it runs as root.

This is actually important for security. Since Git works with files in whatever directory you point it at, running it as root all the time would be risky — a malicious repo could try to overwrite system files. Running as a normal user keeps the damage contained to that user's stuff.

On Git servers (where you host repos for a team), people usually create a dedicated `git` user with a restricted shell. That user owns the bare repositories and can't actually log in interactively. It keeps things locked down.

Managing the Service

Git itself isn't a service — you just use it when you need it and it exits. But if you want to host Git repos on a server, there's `git-daemon` which serves repos over the `git://` protocol:

```
git daemon --reuseaddr --base-path=/srv/git/ /srv/git/
```

If your system uses systemd (most modern distros do), you can manage it like any other service:

```
sudo systemctl start gitdaemon
sudo systemctl stop git-daemon
sudo systemctl status git-
daemon
```

How Updates Work

Updates come through the package manager, same as everything else:

```
sudo apt update sudo
apt upgrade git
```

When someone finds a vulnerability in Git, the security team writes a patch and coordinates with the distro maintainers so Ubuntu, Fedora, Debian, etc. all get the fix around the same time. If you're running `apt upgrade` regularly you'll have the patch within days. The reason this works is because Git is open source. The distro people can read the patch themselves and confirm it actually fixes the problem before they push it out to millions of machines. Compare that to proprietary software where you just download an update and hope it does what it says.

Part C — The FOSS Ecosystem (Units 3 & 4)

What Git Depends On

Git is written in C, so it needs a C compiler (GCC or Clang) and some libraries:

Library	What it does
zlib	Compresses Git objects to save disk space
libcurl	Handles HTTP/HTTPS connections for push, pull, fetch
OpenSSL	Crypto stuff and secure connections
libpcre2	Regular expressions for things like `git grep`
Perl	Some Git sub-commands (like `git send-email`) are actually Perl scripts
GNU Make	The build system if you're compiling from source

All open source. The whole stack underneath Git is built on other people's freely shared work.

What Git Made Possible

This is the part that's kind of mind-blowing when you think about it. Git launched in 2005 and since then it's basically spawned an entire industry:

- **GitHub** came along in 2008 and turned Git into something social. You could follow projects, submit pull requests, file issues. It made open-source

contribution accessible to anyone, not just hardcore mailing-list people. GitHub now has over 330 million repositories. Microsoft bought it in 2018 for $7.5 billion.

- **GitLab** does a lot of what GitHub does, but you can also self-host it. Companies that don't want their code on someone else's servers use it a lot. It also has builtin CI/CD, which GitHub only added later.
- **Gitea and Forgejo** are lightweight alternatives for people who want to host their own Git platform without the heavyweight setup of GitLab.
- **Git LFS** is an extension for handling big binary files (videos, datasets, 3D models) that Git wasn't originally designed for.
- **libgit2** is a C library that reimplements Git's internals so that other apps (like VS Code or other IDEs) can interact with Git repos without running the `git` command-line tool.

How It Connects to the LAMP Stack

Git isn't directly part of the LAMP stack (Linux, Apache, MySQL, PHP). But here's the thing — every part of the LAMP stack is version-controlled with Git. The Linux kernel's source is in a Git repo. Apache's code is in Git. MySQL,
PHP, Python, Ruby — all of them use Git. And modern deployment pipelines (CI/CD) trigger automatic builds and deployments based on Git commits and merges. So while Git isn't in the stack itself, it's the tool through which the entire stack gets developed, tested, and shipped.

Who Runs Git?

No company owns Git. There's no "Git Inc" anywhere. The project is maintained by developers who communicate through a mailing list (git@vger.kernel.org). Junio C Hamano took over as lead maintainer from Torvalds pretty early, back in 2005, and he's still doing it. The way contributions work is almost funny compared to how most modern projects operate. People email patches. Those patches get reviewed on the mailing list. Junio merges the ones that pass review. No pull requests, no fancy web UI. Just email.

On the legal side, Git is part of the Software Freedom Conservancy, which handles administrative stuff. And there's a conference called Git Merge where contributors meet up.

Honestly it kind of blows my mind that a tool used by millions of developers worldwide is run by a small group of people doing code review over email. But 20 years in, it clearly works.

Part D — Open Source vs Proprietary (Critical Analysis)

The closest proprietary competitor to Git is probably **Perforce Helix Core**. It's a centralized version control system that's popular in the gaming industry and some large enterprises.

What we're comparing	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Free. No license fees, no user pricing, nothing.	Free for up to 5 users. per- After that, enterprise pricing — we're talking thousands of dollars per seat per year.
Security	Anyone can read the source code and audit it. Bugs get found and reported by the community.	Source is closed. You're trusting Perforce's internal security team.
Support	Community support — mailing lists, Stack Overflow, docs. No SLA. can buy third-party if you want.	Dedicated support with SLAs, account managers, 24/7 help. You're paying You for it, but it's there. support
Freedom to modify	GPLv2 means you can change anything and redistribute it.	You get what Perforce gives you. Can't modify it, can't extend it.
Who's in control	An independent maintainer and an open mailing list. No single company decides the direction.	Perforce Software Inc. They decide everything — features, pricing, support terms.
Big binary files	Git really struggles with binaries. Git LFS helps but adds its own headaches.	Perforce handles huge binaries natively. This is literally why game studios use it.
Architecture	Distributed — every clone has the full history.	Centralized — you need the server for most things. Works offline.

So which would I actually recommend?

Unless you're a AAA game studio with terabytes of textures and 3D models, Git wins. Not even a close contest. It costs nothing, it's faster than centralized tools for most things, working offline is a huge advantage, and the ecosystem around it is massive.

GitHub, GitLab, CI/CD, all of it is built around Git. Startups, students, open-source projects, normal software teams. Git.

Perforce really only makes sense for large game development, and even there it's losing ground. Studios like Ubisoft or Epic need it for handling massive binary files that Git chokes on. But Git LFS is getting better, and more game companies are actually switching over.

Would I contribute to Git?

I'd like to, yeah. Probably not by writing C code anytime soon. The codebase is maintained by people who work on the Linux kernel and I'd be way out of my depth. But there are other ways to contribute. Documentation, bug reports, testing new releases. I use Git every day so it makes sense to give something back.

Part E — Shell Scripts (Unit 2 & 5)

Script 1 — System Identity Report

File: `system_identity.sh`

This one's basically a welcome screen for the terminal. When you run it, it grabs a bunch of system info — the Linux distro name, kernel version, who's logged in, the home directory, how long the system's been up, and the current date and time — and prints it in a formatted box.

I used command substitution (`$()`) to capture the output of commands like `uname r` and `whoami`, stored them in variables, and then printed them with `echo`. To get the distro name, I used `grep` on `/etc/os-release` and `cut` to pull out just the value. There's also a note about the GPLv2 license at the bottom since the OS itself (Linux) is GPLv2-licensed.

Script 2 — FOSS Package Inspector

File: `foss_inspector.sh`

This script checks whether a given open-source package is installed. By default it checks for `git`, but you can pass any package name as an argument. It tries `dpkg` first (for Debian/Ubuntu) and falls back to `rpm` (for Fedora/Red Hat).

If the package is installed, it shows the version, maintainer, and description. Then there's a `case` statement at the end that matches the package name and prints a short note about the philosophy behind that particular project — like why Git was created, or what makes Firefox different from other browsers.

Script 3 — Disk and Permission Auditor

File: `disk_auditor.sh`

This one has an array of important directories (`/etc`, `/var/log`, `/home`, `/usr/bin`, `/tmp`) and loops through them with a `for` loop. For each one, it checks if it exists, then uses `ls -ld` and `awk` to get permissions, owner, and group, and `du -sh` to get the total size. Everything gets printed in a table using `printf`.

After the main loop, it specifically checks for Git's config files — both `~/.gitconfig` and `~/.config/git/` — and reports whether they exist, what their permissions are, and who owns them. It also checks where the `git` binary lives and what permissions it has.

Script 4 — Log File Analyzer

File: `log_analyzer.sh`

You give this script a log file path and optionally a keyword (defaults to "error"). It reads the file line by line using a `while read` loop, checks each line for the keyword using `grep -iq`, and counts matches with a counter variable and arithmetic expansion (`$(())`).

If the file doesn't exist, there's a retry loop that asks you to type in a different path — that's the do-while style retry the assignment asked for. At the end, it prints how many lines were scanned, how many matched, and then uses `grep | tail -5` to show the last five matching lines.

Script 5 — Open Source Manifesto Generator

File: `manifesto_generator.sh`

This is the creative one. It asks you three questions using `read -p` — your favourite open-source tool, what freedom means to you, and what you'd build if you could share it freely. Then it takes your answers, builds a few paragraphs around them using string concatenation, and writes the whole thing to a file using `>` and `>>`.

The filename includes your username and a timestamp so you don't overwrite previous manifestos. There's also a block of comments at the top explaining how bash aliases work — like `alias ll='ls -la'` — and why they're mostly useful in interactive shells rather than scripts.

Conclusion

Before this project I never thought about Git as anything other than a command. `git add`, `git commit`, `git push`. Just muscle memory. I didn't know why it was distributed or what the hashes were for or who even made those decisions.

Well, now I know. One person made most of those decisions, and he was angry about a licensing dispute, and it took him two weeks. That's funny and impressive at the same time. But the design choices weren't random. Using hashes, making it distributed, optimizing for speed. Each of those came from a real problem Torvalds had experienced with the tools before Git.

The license section surprised me. Before this I assumed licenses were just legal text you slap onto a project and never look at again. Turns out the difference between GPL and MIT has actual consequences for what happens to your code in the future. Whether some company can take it and close it off or not. That's not trivial.

The ethics questions were hard. I don't have firm answers for most of them and I'm not sure anyone does. Whether everything should be open source, whether big companies are being fair to open-source communities. The more I read about it the less certain I got.

The scripts weren't complicated, but writing them was useful. I got more comfortable with bash, especially loops and conditionals. And commenting code properly. I used to skip comments but after writing five scripts where every line needs one, I'm a lot more disciplined about it now.

I guess the big takeaway for me is that open source isn't just some technical licensing thing. It's people choosing to share their work because they think it makes things better for everyone. Git is probably the best proof that they're right.